Contents

1 The `msleep` dataset

This lab uses the `msleep` dataset, which you can download using the `download.file()` function and link below. Make sure that the link is on a single line, so that R can read the link properly. Alternatively, you may also navigate to the link and save the file directly to your computer.

```
download.file("https://raw.githubusercontent.com/genomicsclass/dagdata/master/inst/extda
  destfile = "msleep_ggplot2.csv")
```

Once downloaded, you need to import the data into R, with a use of one of the `read` functions. Which one? The lab sheet suggests `read.csv()`. How can you conclude by yourself what to use? Again, examine the data.

In this case, the file provided has a `.csv` extension, short for *comma-separated values*. Opening the file, you can also see that each row corresponds to one observation, and each feature is separated by a comma. There is also a header row, so set the `header` parameter accordingly.

In the previous lab, I also mentioned that `read.csv()` is the same as `read.table()`. You can in fact use `read.table()`, just change the `sep` parameter.

```
msleep <- read.csv("msleep_ggplot2.csv", header=T)  # equivalent to below
msleep <- read.table("msleep_ggplot2.csv", header=T, sep=",")
```

1.1 Dimension (`dim()`), summary (`summary()`), and head (`head()`) of `msleep`

The lab sheet asks us to use the following functions.

```
dim(msleep)
```

```
## [1] 83 11
```

```
summary(msleep)
```

```
##      name              genus               vore              order
##  Length:83          Length:83          Length:83          Length:83
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
##  conservation        sleep_total      sleep_rem      sleep_cycle
```

```
##  Length:83          Min.   : 1.90   Min.   :0.100   Min.   :0.1167
##  Class :character   1st Qu.: 7.85   1st Qu.:0.900   1st Qu.:0.1833
##  Mode  :character   Median :10.10   Median :1.500   Median :0.3333
##                     Mean   :10.43   Mean   :1.875   Mean   :0.4396
##                     3rd Qu.:13.75   3rd Qu.:2.400   3rd Qu.:0.5792
##                     Max.   :19.90   Max.   :6.600   Max.   :1.5000
##                                     NA's   :22      NA's   :51
##      awake          brainwt            bodywt
##  Min.   : 4.10   Min.   :0.00014   Min.   :   0.005
##  1st Qu.:10.25   1st Qu.:0.00290   1st Qu.:   0.174
##  Median :13.90   Median :0.01240   Median :   1.670
##  Mean   :13.57   Mean   :0.28158   Mean   : 166.136
##  3rd Qu.:16.15   3rd Qu.:0.12550   3rd Qu.:  41.750
##  Max.   :22.10   Max.   :5.71200   Max.   :6654.000
##                  NA's   :27
```

```
head(msleep)
```

```
## # A tibble: 6 x 11
##   name    genus vore  order conservation sleep_total sleep_rem sleep_cycle aw
##   <chr>   <chr> <chr> <chr> <chr>              <dbl>     <dbl>       <dbl> <d
## 1 Cheetah Acin~ carni Carn~ lc                  12.1      NA        NA      1
## 2 Owl mo~ Aotus omni  Prim~ <NA>                17         1.8      NA
## 3 Mounta~ Aplo~ herbi Rode~ nt                  14.4       2.4      NA
## 4 Greate~ Blar~ omni  Sori~ lc                  14.9       2.3       0.133
## 5 Cow     Bos   herbi Arti~ domesticated         4         0.7       0.667  2
## 6 Three-~ Brad~ herbi Pilo~ <NA>                14.4       2.2       0.767
## # i 2 more variables: brainwt <dbl>, bodywt <dbl>
```

From the `dim()` output, we can see that there are 83 rows and 11 columns.

The output of `summary()` gives us some summary statistics of the data. You can see that it lists all the feature names, and where possible, numerical statistics on the feature. For example, `sleep_total` has a minimum of 1.9, a median of 10.10, a mean of 10.43, and a max of 19.90. For features that are names or characters (strings), `summary()` cannot provide a mean, median, etc. as those are numerical statistics.

The `head()` function simply gives the first few rows of the passed data frame. You can optionally pass it a second number to explicitly determine how many rows from the top it should take. Again, you can see this information with the `?` operator.

1.2 Selecting features (`dplyr`'s `select()`)

Most of these exercises are fairly simple. If you face an error doing this task, make sure you have imported `dplyr` with `library()`.

```r
library('dplyr')
select(msleep, name, sleep_total)
select(msleep, -(name:conservation))  # remove the character type data
select(msleep, -(1:4))               # remove the first four columns
msleep %>% select(sleep_total: awake)
msleep %>% select(starts_with("sl"))
```

Note the use of the pipe operator %>% is valid even with only a single verb.

1.3 Filtering observations (dplyr's `filter()`)

Again, this is an exercise of specifying the criteria to filter by accurately.

```r
filter(msleep, sleep_total > 16)
filter(msleep, sleep_total > 16, bodywt > 1)
msleep %>% filter(order %in% c("Perissodactyla", "Primates"))
```

1.4 Pipe operator (%>%)

The pipe operator allows for chaining of functions. It passes the left object as the first input of the right function. If you want to spread it across multiple lines, the pipe operator should be at the end of the line.

```r
msleep %>% select(name, sleep_total) %>%    # <- pipe operator must be here
↪  if line break
  head(6)

# Equivalently, on a single line, without the pipe operator.
# Harder to understand due to its nested nature.
head(select(msleep, name, sleep_total), 6)
```

1.5 Sorting data (dplyr's `arrange()`)

```r
msleep %>% select(order, sleep_total, brainwt) %>%
  arrange(sleep_total) %>% # desc(sleep_total) if required
  filter(sleep_total > 16)
```

```
## # A tibble: 8 x 3
##   order           sleep_total  brainwt
##   <chr>                 <dbl>    <dbl>
## 1 Rodentia               16.6  0.0057
```

```
## 2 Primates                17    0.0155
## 3 Cingulata               17.4  0.0108
## 4 Didelphimorphia         18    0.0063
## 5 Cingulata               18.1  0.081
## 6 Didelphimorphia         19.4 NA
## 7 Chiroptera              19.7  0.0003
## 8 Chiroptera              19.9  0.00025
```

One thing to note about this chaining of operations is that the order of operations may matter for large datasets. In this case, it does not matter significantly whether the dataset is sorted or filtered first. One erroneous example, where sorting by an unselected feature causes an error:

```
# cannot sort by brainwt, since select(msleep, sleep_total)
# only contains sleep_total, and not brainwt.
msleep %>% select(sleep_total) %>%
  arrange(brainwt)
```

In general, if you face an unexpected error or output, **break down your data pipeline and see the output after each step**. That should give you a good idea of where your code went wrong.